



INSTITUTO POLIRTECNICO NACIONAL
ESCUELA SUPERIOR DE COMPUTO
PARADIGMAS DE PROGRAMACIÓN



INTEGRANTES:

Collazo Hernández José Antonio

Cruz Vargas Iker Gael

Santos Fuentes María Fernanda

PROFESORA:

Guadalupe Ana Gabriela Martínez Hernández

GRUPO:

3CV4

1. Recuperación del Trabajo Previo

• Análisis de requerimientos.

A partir de las necesidades operativas clínicas se consideraron 11 requerimientos para nuestro proyecto:

1. **Mostrar servicios y especialidades:** El sistema debe presentar al paciente el catálogo actualizado de especialidades y servicios ofrecidos.
2. **Registro de cuenta del paciente:** Módulo habilitado para la creación, autenticación y gestión de cuentas de usuario.
3. **Programar cita:** Funcionalidad para agendar una consulta vinculando al paciente, especialista, fecha y hora.
4. **Cobro y registro de anticipo:** Procesamiento y registro de un pago inicial o anticipo para formalizar la reserva.
5. **Visualización de agenda médica para el personal:** Interfaz orientada al personal clínico (médicos y enfermeros) para revisar la programación de consultas.
6. **Reprogramación y cancelación de cita:** Permite modificar el día y hora o cancelar definitivamente una cita registrada.
7. **Módulo de donativos:** Sección habilitada para recibir e ingresar aportaciones voluntarias destinadas a la clínica.
8. **Recordatorio de citas:** Envío automatizado de avisos para notificar al paciente sobre sus citas próximas.
9. **Disponibilidad de cita:** Módulo de consulta en tiempo real de los bloques de horario libres de cada profesional.
10. **Generación de comprobantes de citas:** Emisión automática de comprobantes digitales que respaldan el agendamiento y pago.
11. **Consulta de historial de citas:** Registro histórico que permite al usuario o médico revisar la trayectoria de consultas pasadas.

• Roles y actores.

Con base en el análisis funcional del sistema, se definen tres actores fundamentales:

Paciente: Usuario final que interactúa con la plataforma para consultar la oferta médica, registrarse, verificar disponibilidad, describir brevemente sus síntomas, agendar citas, realizar el pago de anticipo, recibir comprobantes e ingresar a su historial.

Personal Médico (Médicos y Enfermeros): Usuarios encargados de la atención médica que consultan la agenda operativa en tiempo real para verificar los horarios y pacientes asignados a su jornada.

Sistema / Módulo de Gestión y Pagos: Entidad automatizada que procesa las solicitudes, valida el cumplimiento de anticipos, ejecuta el bloqueo concurrente de horarios, gestiona la memoria de sesión y emite las notificaciones e impresos digitales.

• Casos de uso.

El flujo operativo principal para la programación de consultas en SanitasFlow comprende los siguientes 10 casos de uso:

- I. **Agendar cita:** Proceso mediante el cual el paciente inicia la solicitud de atención en la clínica.
- II. **Selección de especialidad:** Filtrado del catálogo según la área médica solicitada.
- III. **Selección de un médico o especialista:** Elección del médico adscrito a la especialidad seleccionada.
- IV. **Consulta de disponibilidad:** Verificación del calendario de atención del médico elegido.
- V. **Elección del día y hora:** Selección del bloque de horario deseado por el paciente.
- VI. **Breve síntomas del paciente:** Registro de una breve descripción clínica del motivo de consulta.
- VII. **Confirmación de anticipo:** Despliegue del desglose de cobro del anticipo requerido para apartar la cita.
- VIII. **Validación del pago:** Verificación y acreditación del importe transferido.
- IX. **Bloqueo del horario seleccionado y confirmación:** Asignación exclusiva del horario en el sistema para evitar que otro usuario lo seleccione.
- X. **Envío de un comprobante digital:** Generación y transmisión del comprobante electrónico al paciente

• Elección y evaluación de los lenguajes de programación.

Para el desarrollo multi-paradigma del sistema SanitasFlow se seleccionaron tres lenguajes: Python, Java y C++. La evaluación de sus características clave en el diseño del lenguaje se desglosa a continuación:

Sintaxis: Java y C++ utilizan una estructura delimitada por bloques entre llaves ({}), evitando ambigüedades en la ejecución. Por otro lado, Python implementa una sangría/indentación limpia que favorece la legibilidad rápida y reduce la cantidad de líneas de código.

Semántica: Java y C++ posibilitan el comportamiento polimórfico de objetos y módulos, adaptando la ejecución según el tipo de usuario o actor en el sistema.

Tipado: El control estricto de tipos impide la compilación o ejecución errónea si se intentan ingresar valores de tipo texto en campos numéricos o de tiempo, tales como el monto del anticipo o la hora asignada.

Manejo de Memoria: Java integra un recolector de basura (Garbage Collector) automático que limpia datos temporales de la memoria RAM una vez que el usuario cierra sesión o concluye la consulta de disponibilidades, manteniendo la velocidad del sistema. Por su parte, C++ brinda un control eficiente de bajo nivel.

Estructura de Control: Las estructuras condicionales y lógicas evalúan la cobertura del anticipo para bloquear definitivamente la cita o liberar el horario en caso de impago. Asimismo, garantizan la exclusión mutua para que dos usuarios no puedan apartar al mismo doctor en la misma hora.

2. Selección de una Funcionalidad

1. Qué problema resuelve y quién la utiliza.

Garantiza la integridad en la asignación de horarios en la clínica, evitando la duplicidad de citas (solapamiento o sobreventa de un mismo bloque de tiempo con un médico). Asimismo, asegura que la reserva solo pase a estado confirmado una vez que el pago del anticipo ha sido validado correctamente por el sistema

2. Con qué requerimientos y casos de uso se relaciona.

Paciente: Para consultar la disponibilidad real de un médico, seleccionar su horario deseado y recibir la confirmación de su cita.

Sistema (Servidor Backend): Como ejecutor automático de las reglas de negocio, validaciones de seguridad y control de concurrencia.

3. Cuáles son sus entradas, transformaciones y salidas.

4. Por qué es adecuada para el paradigma funcional y qué ventajas o limitaciones presenta frente a una solución imperativa.

3. Diseño de la Solución

Medico

CLASE Medico

Id : Entero

Nombre : Texto (por defecto vacío)

Especialidad : Texto (por defecto vacío)

Correo : Texto (por defecto vacío)

Cita

CLASE Cita

Id : Entero

PacientId : Entero

MedicId : Entero

FechaCita : FechaHora

HorInicio : Duración

HoraFin : Duración

Estado : Texto (por defecto "Pendiente")

C

CitaService

CLASE CitaService PRIVADO _context : AppDbContext

CONSTRUCTOR(context)

_context = context

FUNCION ObtenerCitasPendientesAsync() → Lista<Cita>

devolver ESPERAR (

_context.Citas

FILTRAR (c → c.Estado = "Pendiente")

(Where) CONVERTIR A LISTA

)

MedicosController

CLASE MedicosController HEREDA DE ControllerBase

RUTA "api/medicos"

PRIVADO _context : AppDbContext

PRIVADO _citaService : CitaService

CONSTRUCTOR(context, citaService) // inyección de dependencias

_context = context

_citaService = citaService

// GET api/medicos

FUNCION GetMedicos() → Respuesta HTTP

medicos = ESPERAR _context.Medicos CONVERTIR A LISTA // aquí falló tu error

devolver Ok(medicos) // HTTP 200 + lista en JSON

Program

INICIO

builder = crear constructor de la aplicación

// 1. Base de datos

cadena = leer "DefaultConnection" de appsettings.json

registrar AppDbContext usando MySQL/MariaDB (Pomelo) con cadena

(detecta la versión del servidor conectándose a él)

// 2. Servicios

registrar CitaService (una instancia por petición)

registrar Controladores

registrar explorador de endpoints

app = construir aplicación

app.UsarRedirecciónHTTPS()

app.UsarAutorización()

app.MapearControladores()

app.EJECUTAR()

5. Implementación

La implementación del sistema SanitasFlow se encuentra estructurada en el proyecto backend SanitasFlow.Api bajo el framework .NET 10 (C#), utilizando PostgreSQL mediante Entity Framework Core (Npgsql.EntityFrameworkCore.PostgreSQL) como sistema de gestión de base de datos relacional.

La arquitectura sigue una separación modular responsabilidades compuesta por Contratos (Contracts), Controladores (Controllers), Modelos de Entidades (Models) y Servicios (Services). Se aplica el paradigma funcional en el procesamiento declarativo de consultas mediante expresiones LINQ y lambdas, así como en las funciones puras encargadas del cálculo de bloques de tiempo y la exclusión de solapamientos en la agenda clínica.

4.1. Modelos de Entidades (SanitasFlow.Api/Models)

Representan el modelo del dominio relacional mapped hacia PostgreSQL.

Paciente.cs

```
namespace SanitasFlow.Api.Models;

public class Paciente
{
    public int Id { get; set; }
    public string Nombres { get; set; } = string.Empty;
    public string ApellidoPaterno { get; set; } = string.Empty;
    public string ApellidoMaterno { get; set; } = string.Empty;
    public int Edad { get; set; }
    public string Correo { get; set; } = string.Empty;
    public string PasswordHash { get; set; } = string.Empty;
    public DateTime CreadoEnUtc { get; set; } = DateTime.UtcNow;
    public List<Cita> Citas { get; set; } = [];
}
```

```
namespace SanitasFlow.Api.Models;

public class Cita
{
    public int Id { get; set; }
    public int PacienteId { get; set; }
    public string EspecialidadId { get; set; } = string.Empty;
    public string ProcedimientoId { get; set; } = string.Empty;
    public DateOnly Fecha { get; set; }
    public int InicioMin { get; set; }
    public int FinMin { get; set; }
    public string Motivo { get; set; } = string.Empty;
    public decimal Precio { get; set; }
    public string Estado { get; set; } = "Confirmada";
    public DateTime CreadaEnUtc { get; set; } = DateTime.UtcNow;
    public Paciente Paciente { get; set; } = null!;
    public Especialidad Especialidad { get; set; } = null!;
    public Procedimiento Procedimiento { get; set; } = null!;
    public List<Pago> Pagos { get; set; } = [];
}
```

4.2. Contexto de Datos y Configuración Relacional (SanitasFlow.Api/Models/AppDbContext.cs)

```
using Microsoft.EntityFrameworkCore;

namespace SanitasFlow.Api.Models
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

        public DbSet<Medico> Medicos => Set<Medico>();
        public DbSet<Paciente> Pacientes => Set<Paciente>();
        public DbSet<Especialidad> Especialidades => Set<Especialidad>();
        public DbSet<Procedimiento> Procedimientos => Set<Procedimiento>();
        public DbSet<Cita> Citas => Set<Cita>();
        public DbSet<Pago> Pagos => Set<Pago>();

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            modelBuilder.Entity<Paciente>(entity =>
            {
                entity.HasIndex(p => p.Correo).IsUnique();
                entity.Property(p => p.Correo).HasMaxLength(320);
                entity.Property(p => p.PasswordHash).HasMaxLength(512);
            });

            modelBuilder.Entity<Especialidad>().HasKey(e => e.Id);
            modelBuilder.Entity<Procedimiento>().HasKey(p => p.Id);
            modelBuilder.Entity<Procedimiento>()
                .HasOne(p => p.Especialidad)
                .WithMany(e => e.Procedimientos)
                .HasForeignKey(p => p.EspecialidadId)
                .OnDelete(DeleteBehavior.Restrict);

            modelBuilder.Entity<Cita>(entity =>
            {
                entity.HasIndex(c => new { c.EspecialidadId, c.Fecha });
                entity.Property(c => c.Precio).HasPrecision(10, 2);
                entity.Property(c => c.Estado).HasMaxLength(32);
                entity.HasOne(c => c.Paciente).WithMany(p => p.Citas).HasForeignKey(c => c.PacienteId);
                entity.HasOne(c => c.Especialidad).WithMany().HasForeignKey(c =>
                c.EspecialidadId).OnDelete(DeleteBehavior.Restrict);
                entity.HasOne(c => c.Procedimiento).WithMany().HasForeignKey(c =>
                c.ProcedimientoId).OnDelete(DeleteBehavior.Restrict);
            });

            modelBuilder.Entity<Pago>(entity =>
            {
                entity.Property(p => p.Monto).HasPrecision(10, 2);
            });
        }
    }
}
```

```

        entity.Property(p => p.Monto).HasPrecision(10, 2);
        entity.HasOne(p => p.Cita).WithMany(c => c.Pagos).HasForeignKey(p => p.CitaId);
    });
}
}
}
}

```

4.3. Capa de Servicios y Algoritmos Funcionales (SanitasFlow.Api/Services/CitaService.cs)

Esta clase contiene la lógica principal de la funcionalidad seleccionada. Aplica técnicas del paradigma funcional mediante:

Modelado inmutable de ventanas de atención: Uso de diccionarios de sólo lectura (IReadOnlyDictionary) para mapear la disponibilidad diaria.

Funciones puras y filtros declarativos: Evaluaciones con Any(), Where() e iteraciones sin efectos secundarios colaterales para calcular los intervalos de minutos sin traslapes.

Manejo explícito de concurrencia: Creación transaccional con nivel de aislamiento serializable (IsolationLevel.Serializable) para bloquear el horario en la base de datos.

```

using Microsoft.EntityFrameworkCore;
using SanitasFlow.Api.Models;
using System.Data;

namespace SanitasFlow.Api.Services
{
    public class CitaService
    {
        private readonly AppDbContext _context;

        public CitaService(AppDbContext context)
        {
            _context = context;
        }

        private static readonly IReadOnlyDictionary<DayOfWeek, (int Inicio, int Fin)[]> Horarios =
            new Dictionary<DayOfWeek, (int, int)[]>
            {
                [DayOfWeek.Monday] = [(540, 840), (900, 1080)],
                [DayOfWeek.Tuesday] = [(540, 840), (900, 1080)],
                [DayOfWeek.Wednesday] = [(540, 840), (900, 1080)],
                [DayOfWeek.Thursday] = [(540, 840), (900, 1080)],
                [DayOfWeek.Friday] = [(540, 840), (900, 1080)],
            }
    }
}

```

```

        [DayOfWeek.Saturday] = [(540, 780)],
        [DayOfWeek.Sunday] = []
    };

    public async Task<List<int>> ObtenerHorariosDisponiblesAsync(string especialidadId, string procedimientoId, DateOnly fecha,
CancellationTokens cancellationTokens)
    {
        var procedimiento = await ObtenerProcedimientoAsync(especialidadId, procedimientoId, cancellationTokens);
        var ocupadas = await ObtenerCitasOcupadasAsync(especialidadId, fecha, cancellationTokens);
        return CalcularHorarios(fecha, procedimiento.DuracionMin, ocupadas);
    }

    public async Task<Cita> CrearCitaAsync(int pacienteId, string especialidadId, string procedimientoId, DateOnly fecha, int
inicioMin, string motivo, CancellationTokens cancellationTokens)
    {
        await using var transaction = await _context.Database.BeginTransactionAsync(IsolationLevel.Serializable,
cancellationTokens);
        var procedimiento = await ObtenerProcedimientoAsync(especialidadId, procedimientoId, cancellationTokens);
        var ocupadas = await ObtenerCitasOcupadasAsync(especialidadId, fecha, cancellationTokens);
        var finMin = inicioMin + procedimiento.DuracionMin;

        if (!CalcularHorarios(fecha, procedimiento.DuracionMin, ocupadas).Contains(inicioMin))
        {
            throw new InvalidOperationException("El horario solicitado ya no está disponible o queda fuera del horario de
atención.");
        }

        var cita = new Cita
        {
            PacienteId = pacienteId,
            EspecialidadId = especialidadId,
            ProcedimientoId = procedimientoId,
            Fecha = fecha,
            InicioMin = inicioMin,
            FinMin = finMin,
            Motivo = motivo.Trim(),
            Precio = procedimiento.Precio,
            Estado = "Confirmada"
        };
        _context.Citas.Add(cita);
        _context.Pagos.Add(new Pago
        {
            Cita = cita,
            Monto = procedimiento.Precio,
            ReferenciaExterna = $"simulado-{Guid.NewGuid():N}"
        });
    }

    await _context.SaveChangesAsync(cancellationTokens);
    await transaction.CommitAsync(cancellationTokens);
    return cita;
}

private async Task<Procedimiento> ObtenerProcedimientoAsync(string especialidadId, string procedimientoId,
CancellationTokens cancellationTokens)
{
    return await _context.Procedimientos.AsNoTracking()
        .SingleOrDefaultAsync(p => p.Id == procedimientoId && p.EspecialidadId == especialidadId, cancellationTokens)
        ?? throw new InvalidOperationException("El procedimiento solicitado no existe para esa especialidad.");
}

private async Task<List<Cita>> ObtenerCitasOcupadasAsync(string especialidadId, DateOnly fecha, CancellationTokens
cancellationTokens)
{
    return await _context.Citas
        .Where(c => c.EspecialidadId == especialidadId && c.Fecha == fecha && c.Estado != "Cancelada")
        .ToListAsync(cancellationTokens);
}

private static List<int> CalcularHorarios(DateOnly fecha, int duracionMin, List<Cita> ocupadas)
{

```

```

    {
        if (fecha < DateOnly.FromDateTime(DateTime.Today) || !Horarios.TryGetValue(fecha.DayOfWeek, out var ventanas))
        {
            return [];
        }

        var hoy = fecha == DateOnly.FromDateTime(DateTime.Today);
        var ahoraMin = DateTime.Now.Hour * 60 + DateTime.Now.Minute;
        var resultado = new List<int>();
        foreach (var (inicio, fin) in ventanas)
        {
            for (var hora = inicio; hora + duracionMin <= fin; hora += 15)
            {
                if (hoy && hora <= ahoraMin) continue;
                if (ocupadas.Any(c => hora < c.FinMin && hora + duracionMin > c.InicioMin)) continue;
                resultado.Add(hora);
            }
        }
        return resultado;
    }
}
}

```

4.4. Controladores API (SanitasFlow.Api/Controllers/CitasController.cs)

Exponen los puntos de acceso HTTP RESTful protegidos bajo la autenticación mediante cookies ([Authorize])

```

using System.Security.Claims;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using SanitasFlow.Api.Contracts;
using SanitasFlow.Api.Models;
using SanitasFlow.Api.Services;

namespace SanitasFlow.Api.Controllers;

[ApiController]
[Authorize]
[Route("api/citas")]
public class CitasController(AppDbContext context, CitaService citaService) : ControllerBase
{
    [HttpGet("disponibilidad")]
    public async Task<ActionResult<object>> Disponibilidad([FromQuery] string especialidadId, [FromQuery] string procedimientoId,
        [FromQuery] string fecha, CancellationToken cancellationToken)
    {
        if (!DateOnly.TryParse(fecha, out var fechaCita)) return BadRequest(new { message = "La fecha no es válida." });
        try
        {
            var horas = await citaService.ObtenerHorariosDisponiblesAsync(especialidadId, procedimientoId, fechaCita,

```

```

cancellationToken);
    return Ok(new { horas });
}
catch (InvalidOperationException ex)
{
    return BadRequest(new { message = ex.Message });
}
}

[HttpGet("mias")]
public async Task<ActionResult<List<AppointmentResponse>>> Mias(Cancellation token cancellationToken)
{
    if (!TryGetPatientId(out var pacienteId)) return Unauthorized();
    var citas = await context.Citas.AsNoTracking()
        .Include(c => c.Especialidad).Include(c => c.Procedimiento)
        .Where(c => c.PacienteId == pacienteId)
        .OrderBy(c => c.Fecha).ThenBy(c => c.InicioMin)
        .ToListAsync(cancellationToken);
    return Ok(citas.Select(ToResponse).ToList());
}

[HttpPost]
public async Task<ActionResult<AppointmentResponse>> Create(CreateAppointmentRequest request, Cancellation token

```

```

cancellationToken)
{
    if (!TryGetPatientId(out var pacienteId)) return Unauthorized();
    if (!DateOnly.TryParse(request.Fecha, out var fecha)) return BadRequest(new { message = "La fecha no es válida." });

    try
    {
        var cita = await citaService.CrearCitaAsync(pacienteId, request.EspecialidadId, request.ProcedimientoId, fecha,
request.InicioMin, request.Motivo ?? string.Empty, cancellationToken);
        var response = await context.Citas.AsNoTracking().Include(c => c.Especialidad).Include(c => c.Procedimiento)
            .SingleAsync(c => c.Id == cita.Id, cancellationToken);
        return CreatedAtAction(nameof(GetById), new { id = cita.Id }, ToResponse(response));
    }
    catch (InvalidOperationException ex)
    {
        return Conflict(new { message = ex.Message });
    }
}

private bool TryGetPatientId(out int patientId) => int.TryParse(User.FindFirstValue(ClaimTypes.NameIdentifier), out patientId);

private static AppointmentResponse ToResponse(Cita cita) => new(cita.Id, cita.Especialidad.Nombre, cita.Procedimiento.Nombre,
    cita.Fecha.ToString("yyyy-MM-dd"), cita.InicioMin, cita.FinMin, cita.Precio, cita.Motivo, cita.Estado);

```

4.5. Configuración del Servidor y Middleware (SanitasFlow.Api/Program.cs)

```
using Microsoft.AspNetCore.Authentication.Cookies;
using Microsoft.EntityFrameworkCore;
using SanitasFlow.Api.Models;
using SanitasFlow.Api.Services;

var builder = WebApplication.CreateBuilder(args);

// Registro de PostgreSQL mediante EF Core
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection")
    ?? throw new InvalidOperationException("Falta la cadena de conexión DefaultConnection.");
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseNpgsql(connectionString));

// Configuración de cookies HTTP-Only de autenticación
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.Cookie.Name = "SanitasFlow.Auth";
        options.Cookie.HttpOnly = true;
        options.Cookie.SameSite = SameSiteMode.Lax;
        options.Events.OnRedirectToLogin = context =>
        {
            context.Response.StatusCode = StatusCodes.Status401Unauthorized;
            return Task.CompletedTask;
        }
    });
```

```
});
});
builder.Services.AddAuthorization();

builder.Services.AddScoped<CitaService>();
builder.Services.AddSingleton<PasswordService>();
builder.Services.AddControllers();

var app = builder.Build();

if (app.Environment.IsDevelopment() || app.Configuration.GetValue<bool>("Database:EnsureCreated"))
{
    await using var scope = app.Services.CreateAsyncScope();
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    await db.Database.EnsureCreatedAsync();
    await CatalogSeed.SeedAsync(db);
}

app.UseHttpsRedirection();
app.UseDefaultFiles();
app.UseStaticFiles();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
```

```
app.UseAuthorization();
app.MapControllers();
app.MapFallbackToFile("index.html");

app.Run();
```

5. Pruebas

Para verificar la calidad y el correcto funcionamiento del sistema, se ejecutó un plan de pruebas sobre la interfaz web desarrollada en el frontend, validando la comunicación asíncrona con los servicios backend de la API (.NET 10) y la persistencia de datos en PostgreSQL.

Las pruebas se enfocarán en el flujo completo que realiza un usuario paciente al seleccionar un procedimiento, verificar disponibilidad, procesar el pago de anticipo/cita y visualizar la confirmación en su historial.

5.1. Prueba 1: Consulta del Catálogo de Especialidades y Procedimientos

Objetivo: Verificar que el sistema renderice correctamente la lista de especialidades médicas y sus respectivos procedimientos con precios y tiempos estimados cargados desde la API (/api/catalogo).

Procedimiento:

1. Navegar a la sección de Especialidades
2. Desplegar una especialidad
3. Verificar la visualización de la descripción, precio aproximado en pesos MXN y la duración estimada en minutos de cada procedimiento.

 **Dermatología**

Diagnóstico y tratamiento de la piel, cabello y uñas.

Consulta dermatológica Valoración general de piel, cabello o uñas. Costo aprox. \$700 MXN. · Duración aprox. 30 min	\$700 MXN	Agendar
Revisión de lunares Revisión y seguimiento de lunares o manchas. Costo aprox. \$650 MXN. · Duración aprox. 30 min	\$650 MXN	Agendar
Crioterapia Eliminación de verrugas o lesiones con frío controlado. Costo aprox. \$500 MXN. · Duración aprox. 20 min	\$500 MXN	Agendar
Biopsia de piel Toma de muestra de piel para análisis. Costo aprox. \$1,400 MXN. · Duración aprox. 45 min	\$1400 MXN	Agendar
Tratamiento de acné Valoración y tratamiento dirigido de acné. Costo aprox. \$600 MXN. · Duración aprox. 30 min	\$600 MXN	Agendar

 **Pediatría**

Atención médica para bebés, niñas, niños y adolescentes.

Consulta pediátrica

Valoración general del paciente pediátrico. Costo aprox. \$600 MXN. · Duración aprox. 30 min

\$600 MXN

Agendar

Control de niño sano

Seguimiento de crecimiento y desarrollo. Costo aprox. \$550 MXN. · Duración aprox. 30 min

\$550 MXN

Agendar

Vacunación

Aplicación de esquema de vacunación. Costo aprox. \$400 MXN (precio varía según vacuna). · Duración aprox. 15 min

\$400 MXN

Agendar

Valoración de desarrollo

Evaluación de hitos de desarrollo del menor. Costo aprox. \$700 MXN. · Duración aprox. 40 min

\$700 MXN

Agendar

5.2. Prueba 2: Selección Dinámica de Horarios Disponibles (Algoritmo Funcional)

Objetivo: Comprobar que el endpoint `/api/citas/disponibilidad` filtre correctamente en tiempo real los bloques de horario libres, omitiendo periodos ocupados y el horario de comida de la clínica.

Procedimiento:

1. Seleccionar un procedimiento y presionar "Agendar".
2. Seleccionar una fecha del calendario/chips de fechas.
3. Comprobar que se desplieguen los bloques de tiempo (slots) de 15 en 15 minutos en estado disponible.

Cita general • Consulta general

Valoración médica general. Costo aprox. \$450 MXN. Duración aprox. 30 minutos.

1. Elige el día

VIE 2	SÁB 3	LUN 5	MAR 6	MIÉ 7
JUÉ 8	VIE 9	SÁB 10	LUN 12	MAR 13

2. Elige el horario

9:00 a.m.	9:15 a.m.	9:30 a.m.	9:45 a.m.	10:00 a.m.	10:15 a.m.
10:30 a.m.	10:45 a.m.	11:00 a.m.	11:15 a.m.	11:30 a.m.	11:45 a.m.
12:00 p.m.	12:15 p.m.	12:30 p.m.			

Motivo de tu consulta (opcional)

tengo el cuerpo cortado y fiebre

Continuar al pago

Cancelar

5.3. Prueba 3: Proceso de Pago y Confirmación de la Reserva

Objetivo: Validar la creación de la cita en estado "Confirmada" junto con la emisión simulada de su comprobante y registro en la base de datos PostgreSQL.

Procedimiento:

1. Confirmar el horario deseado y avanzar a la pantalla de pago.
2. Verificar el desglose de resumen (especialidad, fecha, hora exacta y monto a pagar).
3. Completar el formulario y hacer clic en "Confirmar pago y agendar".
4. Redirección automática a la vista de confirmación

Resumen de tu cita

Especialidad: Cita general

Procedimiento: Consulta general

Fecha: sábado, 3 de octubre

Horario: 12:15 p.m. - 12:45 p.m.

Total a pagar: **\$450 MXN**

Datos de pago

Este es un formulario de demostración; no procesa pagos reales.

Nombre en la tarjeta

Número de tarjeta

Vigencia

CVV

Confirmar y pagar

Cancelar



Tu cita quedó confirmada

Especialidad: Cita general

Procedimiento: Consulta general

Fecha: sábado, 3 de octubre

Horario: 12:15 p.m. - 12:45 p.m.

Motivo: tengo el cuerpo cortado y fiebre

Total pagado: \$450 MXN

[Ver mis citas](#)

[Ir al inicio](#)

5.4. Prueba 4: Historial de Citas del Paciente y Persistencia

Objetivo: Verificar que la cita agendada se liste en el panel personal del paciente (bienvenida-paciente.html) y que los datos queden guardados en las tablas relacionales de PostgreSQL.

Procedimiento:

1. Ingresar a la vista "Mis Citas" del paciente.
2. Confirmar la aparición de la nueva cita con la insignia/badge "Confirmada".

Hola, cesar

Aquí puedes ver tus próximas citas y acceder a tus datos personales.

Agendar nueva cita

Mis datos personales

Mis citas

Cita general · Consulta general

sábado, 3 de octubre · 12:15 p.m.

Confirmada

ID	Caso de prueba	Entrada/Vista	Resultado esperado	Resultado obtenido	Estatus
CP-01	Carga de catalogo	especialidades.html	Muestra el catálogo de especialidades y procedimientos con precios desde PostgreSQL.	Datos cargados dinámicamente mediante fetch a /api/catalogo	Exitoso
CP-02	Filtrado de horarios	reservar-cita.html	Despliega únicamente bloques de tiempo libres en la jornada operativa	Filtro funcional genera la lista de minutos excluyendo solapamientos.	Exitoso
CP-03	Agendamiento y pagos	pago-cita.html	Genera la cita en estado Confirmada con su registro transaccional de pago.	Cita y pago simulado creados en transacción con aislamiento Serializable	Exitoso
CP-04	Consulta de historial	Bienvenida-paciente.html	Muestra las citas asociadas únicamente al paciente autenticado con cookie	Colección de citas recuperada y renderizada correctamente	Exitoso

6. Repositorio de GitHub

El código fuente completo del proyecto, los scripts SQL y los módulos cliente se encuentran disponibles en el repositorio oficial:

URL del Repositorio:

<https://github.com/tynon6/Gestor-de-Citas-para-Hospital.git>